Ad Soyad: _____	**Tarih:** _____
Lab 08.5 – OBYS Sıfırdan Geliştirme (Bonus) – *Adım Adım Qt/C++ Masaüstü Uygulama Tutorial*	

Tutorial Hakkında

Bu çalışma kağıdında, dönem boyunca kitapta incelediğimiz OBYS (Öğrenci Bilgi Yönetim Sistemi) uygulamasını **sıfırdan** adım adım kodlayacaksınız. Mevcut lab örneklerinden farklı olarak burada hazır başlangıç kodu **yoktur** — her satırı siz yazacaksınız.
Uygulama 7 aşamada geliştirilecektir:

1. **Veri Sınıfları** – Domain struct'ları ve `std::variant` not türleri
2. **Yardımcı Sınıflar** – Servis katmanı ve kalıcılık yöneticisi
3. **Veri Giriş Formları** – Öğrenci, ders ve not giriş diyalogları (`.ui` dosyalı)
4. **Ana Pencere** – Sekmeli ana pencere, menüler ve tablo görünümü
5. **DTO ve İleri Konular** – JSON dışa aktarım, ayarlar penceresi
6. **Grafikler ve Raporlar** – QtCharts ile istatistik, transkript üretimi
7. **Kurulum ve Dağıtım** – Tema desteği, QSettings, son dokunuşlar

Süre: 6–8 saat (birden fazla oturumda tamamlanabilir) **Gereksinimler:** Qt6 (Widgets, PrintSupport, Charts), CMake 3.21+, C++20

Önemli Notlar

- Her aşamanın sonunda projeyi **derleyip çalıştırın**. Bir aşamayı tamamlamadan diğerine geçmeyin.
- Bu tutorial mevcut lab örneklerinden bağımsızdır; `out/LabExamples/` altındaki dosyaları **kullanmayın**.
- Kodlamaya tamamen boş bir dizinden başlıyorsunuz.
- Her aşama sonundaki **kontrol listesini** eksiksiz tamamlayın.

1 Aşama 0: Proje İskeleti Oluşturma (15 dk)

Ön Bilgi

Her Qt projesi bir `CMakeLists.txt` dosyası ile başlar. Bu dosya projenin adını, kullanılan C++ standardını, Qt bileşenlerini ve kaynak dosyaları tanımlar. İlk aşamada yalnızca boş bir pencere açan minimal bir proje oluşturacağız.

Adım 0.1: Dizin Yapısını Oluşturun

Çalışma dizininizde aşağıdaki yapıyı oluşturun:

```
obys_tutorial/
  CMakeLists.txt
  include/
  src/
  ui/
  resources/
    themes/
```

```
mkdir -p obys_tutorial/{include,src,ui,resources/themes}
cd obys_tutorial
```

Adım 0.2: CMakeLists.txt Dosyasını Yazın

```
cmake_minimum_required(VERSION 3.21)

project(obys_tutorial
    VERSION 1.0.0
    DESCRIPTION "OBYS - Ogrenci Bilgi Yonetim Sistemi"
    LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)
set(CMAKE_AUTOUIC ON)

find_package(Qt6 6.5 REQUIRED COMPONENTS
    Widgets PrintSupport Charts)

qt_standard_project_setup()

# Kaynak dosyalar (her asamada genisleyecek)
set(OBYS_SOURCES
    src/main.cpp
)

qt_add_executable(obys_tutorial ${OBYS_SOURCES})

target_include_directories(obys_tutorial
    PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}/include
        ${CMAKE_CURRENT_SOURCE_DIR}/src
```

```
        ${CMAKE_CURRENT_SOURCE_DIR}/ui
)

target_link_libraries(obys_tutorial
    PRIVATE
        Qt6::Widgets
        Qt6::PrintSupport
        Qt6::Charts
)

if(MSVC)
    target_compile_options(obys_tutorial PRIVATE /W4)
else()
    target_compile_options(obys_tutorial PRIVATE
        -Wall -Wextra -Wpedantic)
endif()
```

Adım 0.3: Minimal main.cpp Yazın

`src/main.cpp` dosyasını oluşturun:

```
#include <QApplication>
#include <QMainWindow>

int main(int argc, char* argv[]) {
    QApplication uygulama(argc, argv);

    QCoreApplication::setOrganizationName("KTU");
    QCoreApplication::setApplicationName("OBYS Tutorial");
    QCoreApplication::setApplicationVersion("1.0.0");

    QMainWindow pencere;
    pencere.setWindowTitle("OBYS Tutorial");
    pencere.resize(900, 600);
    pencere.show();

    return uygulama.exec();
}
```

Adım 0.4: Derleyin ve Çalıştırın

```
cmake -B build
cmake --build build -j
./build/obys_tutorial
```

Kontrol Listesi – Aşama 0

- ☐ Boş pencere açılıyor mu?
- ☐ Pencere başlığı “OBYS Tutorial” olarak görünüyor mu?
- ☐ Derleme uyarısız tamamlanıyor mu?

2 Aşama 1: Veri Sınıfları (45 dk)

Ön Bilgi

OBYS sisteminin temelini üç veri yapısı oluşturur: `Ogrenci`, `Ders` ve `DersKaydi`. Not sistemi sayısal (0–100), harf notu (AA, BA, ...) veya muaf olmak üzere üç farklı türü desteklemelidir. C++17'nin `std::variant` yapısı bu çeşitliliği tip güvenli biçimde modellemek için idealdir. Ayrıca verilerin dosyaya yazılıp okunabilmesi için `QDataStream` operatörleri tanımlayacağız.

Adım 1.1: Not Türleri (`std::variant`)

`include/obys_domain.h` dosyasını oluşturun. Dosyanın başına `#ifndef` koruması ekledikten sonra şu yapıları tanımlayın:

```
#ifndef OBYS_DOMAIN_H
#define OBYS_DOMAIN_H

#include <QDataStream>
#include <QString>
#include <variant>

// Not turleri -- std::variant ile tip guvenli birlestirme
struct SayisalNot { double deger{0.0}; };
struct HarfNotu   { QString harf; };
struct MuafNot    { };

using NotTuru = std::variant<SayisalNot, HarfNotu, MuafNot>;
```

std::variant Neden Kullanıyoruz?

Geleneksel yaklaşımda sınıf hiyerarşisi ve sanal fonksiyonlar kullanılırdı. `std::variant` ile derleyici tüm türlerin ele alındığını **derleme zamanında** garanti eder. Ayrıca yığın (heap) tahsisi gerektirmez; bu da performans ve bellek güvenliği açısından avantajlıdır.

Adım 1.2: Domain Struct'ları

Aynı dosyada temel yapıları tanımlayın:

```
struct Ogrenci {
    int     id{-1};
    QString numara;       // Ogrenci numarasi (ornegin "2023001")
    QString adSoyad;      // Ad ve soyad
};

struct Ders {
    int     id{-1};
    QString kod;          // Ders kodu (ornegin "BILB2000")
    QString ad;           // Ders adi
    int     kredi{0};     // Kredi sayisi
};

struct DersKaydi {
    int     ogrenciId{-1};
    int     dersId{-1};
    NotTuru notu{SayisalNot{0.0}};
};
```

Görev 1.1 – Sorular (5 dk)

Aşağıdaki soruları yanıtlayın:

a) `int id{-1}` ifadesindeki `{-1}` ne anlama gelir? Neden varsayılan değer olarak -1 seçilmiştir?

_____

b) `DersKaydi` içinde neden doğrudan `Ogrenci` nesnesi değil de `ogrenciId` (yabancı anahtar) tutulmuştur?

_____

c) `std::variant<SayisalNot, HarfNotu, MuafNot>` yerine `std::any` kullansaydık ne gibi dezavantajlar olurdu?

_____

Adım 1.3: QDataStream Serileştirme Operatörleri

Verilerimizi ikili (binary) dosyaya yazıp okuyabilmek için `operator«` ve `operator»` aşırı yüklemelerini tanımlamamız gerekir. Aynı `obys_domain.h` dosyasında devam edin:

```
// Ogrenci serilestirme
inline QDataStream& operator<<(QDataStream& s,
                               const Ogrenci& o) {
    s << static_cast<qint32>(o.id)
      << o.numara << o.adSoyad;
    return s;
}

inline QDataStream& operator>>(QDataStream& s, Ogrenci& o) {
    qint32 id = -1;
    s >> id >> o.numara >> o.adSoyad;
    o.id = id;
    return s;
}

// Ders serilestirme
inline QDataStream& operator<<(QDataStream& s,
                               const Ders& d) {
    s << static_cast<qint32>(d.id)
      << d.kod << d.ad
      << static_cast<qint32>(d.kredi);
    return s;
}

inline QDataStream& operator>>(QDataStream& s, Ders& d) {
    qint32 id = -1, kredi = 0;
    s >> id >> d.kod >> d.ad >> kredi;
    d.id    = id;
    d.kredi = kredi;
    return s;
}
```

Adım 1.4: NotTuru Variant Serileştirme

`NotTuru` için serileştirme biraz daha karmaşıktır: önce hangi alternatifin aktif olduğunu belirten bir etiket (tag) yazılır, ardından ilgili veri yazılır.

```cpp
// NotTuru: once etiket (0=Sayisal, 1=Harf, 2=Muaf)
inline QDataStream& operator<<(QDataStream& s,
                               const NotTuru& n) {
    const quint8 etiket = static_cast<quint8>(n.index());
    s << etiket;
    std::visit([&s](const auto& deger) {
        using T = std::decay_t<decltype(deger)>;
        if constexpr (std::is_same_v<T, SayisalNot>) {
            s << deger.deger;
        } else if constexpr (std::is_same_v<T, HarfNotu>) {
            s << deger.harf;
        }
        // MuafNot icin ek veri yok
    }, n);
    return s;
}

inline QDataStream& operator>>(QDataStream& s, NotTuru& n) {
    quint8 etiket = 0;
    s >> etiket;
    switch (etiket) {
        case 0: {
            SayisalNot v;
            s >> v.deger;
            n = v;
            break;
        }
        case 1: {
            HarfNotu v;
            s >> v.harf;
            n = v;
            break;
        }
        case 2:
            n = MuafNot{};
            break;
        default:
            s.setStatus(QDataStream::ReadCorruptData);
            break;
    }
    return s;
}
```

Adım 1.5: DersKaydi Serileştirme ve NotDTO

```cpp
inline QDataStream& operator<<(QDataStream& s,
                               const DersKaydi& k) {
    s << static_cast<qint32>(k.ogrenciId)
      << static_cast<qint32>(k.dersId)
      << k.notu;
    return s;
}

inline QDataStream& operator>>(QDataStream& s,
                               DersKaydi& k) {
    qint32 ogr = -1, drs = -1;
    s >> ogr >> drs >> k.notu;
    k.ogrenciId = ogr;
```

```
    k.dersId     = drs;
    return s;
}

// Dialog ile ana pencere arasinda veri tasimak icin DTO
struct NotDTO {
    int      ogrenciId{-1};
    int      dersId{-1};
    NotTuru notu{SayisalNot{0.0}};
};

#endif // OBYS_DOMAIN_H
```

Görev 1.2 – Sorular (5 dk)

a) Serileştirmede `static_cast<qint32>(o.id)` neden gereklidir? Doğrudan `int` yazsaydık ne olurdu?

_____

b) `n.index()` fonksiyonu ne döndürür ve neden etiket olarak kullanıyoruz?

_____

c) `NotDTO` ile `DersKaydi` arasındaki fark nedir? Neden ayrı bir yapı tanımladık?

_____

Kontrol Listesi – Aşama 1

- ☐ `include/obys_domain.h` dosyası tamamlandı
- ☐ Tüm struct'lar (`Ogrenci`, `Ders`, `DersKaydi`) tanımlı
- ☐ `NotTuru` variant ve üç alternatifi tanımlı
- ☐ QDataStream operatörleri tüm struct'lar için yazıldı
- ☐ `NotDTO` tanımlı
- ☐ Proje derleniyor (`cmake -build build`)

3 Aşama 2: Yardımcı Sınıflar – Servis ve Kalıcılık (45 dk)

Ön Bilgi

Veri sınıflarımız hazır; şimdi bu verileri yönetecek ve dosyaya kaydedecek/okuyacak sınıflara ihtiyacımız var. `OBYSServis` bellek içi veri deposu olarak görev yapacak, `PersistenceManager` ise ikili (binary) dosya okuma/yazma işlemlerini üstlenecek.

Adım 2.1: OBYSServis Sınıfı

`include/obys_servis.h` dosyasını oluşturun:

```
#ifndef OBYS_SERVIS_H
#define OBYS_SERVIS_H

#include "obys_domain.h"
#include <QList>

class OBYSServis {
public:
    OBYSServis() = default;

    // Veri erisimi
    QList<Ogrenci>   tumOgrenciler() const
        { return m_ogrenciler; }
    QList<Ders>      tumDersler()    const
        { return m_dersler; }
    QList<DersKaydi> tumNotlar()     const
        { return m_notlar; }

    // Ekleme
    void ogrenciEkle(const Ogrenci& o)
        { m_ogrenciler.append(o); }
    void dersEkle(const Ders& d)
        { m_dersler.append(d); }
    void dersKaydiEkle(const DersKaydi& k)
        { m_notlar.append(k); }

    // DTO'dan not ekle (dialog akisi icin)
    void notEkle(const NotDTO& dto) {
        DersKaydi k;
        k.ogrenciId = dto.ogrenciId;
        k.dersId    = dto.dersId;
        k.notu      = dto.notu;
        m_notlar.append(k);
    }

    // Temizle (yuklemeden once cagirilir)
    void temizle() {
        m_ogrenciler.clear();
        m_dersler.clear();
        m_notlar.clear();
    }

    // Ornek verilerle doldur
    void ornekVerileriYukle() {
        temizle();
        m_ogrenciler = {
```

```
            {1, "2023001",
             QStringLiteral("Ayşe Yılmaz")},
            {2, "2023002",
             QStringLiteral("Mehmet Demir")},
            {3, "2023003",
             QStringLiteral("Zeynep Kaya")},
        };
        m_dersler = {
            {10, "BILB2000",
             QStringLiteral(
                 "Yazılım Geliştirme II"), 4},
            {11, "BILB2001",
             QStringLiteral(
                 "Veri Yapıları"), 4},
        };
    }

private:
    QList<Ogrenci>   m_ogrenciler;
    QList<Ders>      m_dersler;
    QList<DersKaydi> m_notlar;
};

#endif // OBYS_SERVIS_H
```

İpucu: QStringLiteral Kullanımı

Daha temiz bir alternatif olarak `QStringLiteral("Ayşe Yılmaz")` kullanabilirsiniz. UTF-8 kaynak dosyanız doğru kodlanmışsa Qt Türkçe karakterleri sorunsuz işleyecektir.

Adım 2.2: PersistenceManager Başlık Dosyası

`src/persistence_manager.h` dosyasını oluşturun:

```
#ifndef PERSISTENCE_MANAGER_H
#define PERSISTENCE_MANAGER_H

#include <QString>
#include <cstdint>

class OBYSServis;

class PersistenceManager {
public:
    // Dosya imzasi: 'O','B','Y','S' = 0x4F425953
    static constexpr quint32 MAGIC   = 0x4F425953u;
    static constexpr quint32 VERSION = 1u;

    PersistenceManager() = default;

    bool kaydet(const QString& dosyaYolu,
                OBYSServis& servis);
    bool yukle(const QString& dosyaYolu,
               OBYSServis& servis);

    QString sonHata() const { return m_sonHata; }

private:
    QString m_sonHata;
```

```
};

#endif // PERSISTENCE_MANAGER_H
```

Adım 2.3: PersistenceManager Uygulama Dosyası

`src/persistence_manager.cpp` dosyasını oluşturun:

```
#include "persistence_manager.h"
#include "obys_domain.h"
#include "obys_servis.h"

#include <QCoreApplication>
#include <QDataStream>
#include <QFile>
#include <QFileInfo>

bool PersistenceManager::kaydet(const QString& dosyaYolu,
                                OBYSServis& servis) {
    m_sonHata.clear();
    QFile dosya(dosyaYolu);
    if (!dosya.open(QIODevice::WriteOnly
                    | QIODevice::Truncate)) {
        m_sonHata = QString("Dosya acilamadi: %1")
                        .arg(dosya.errorString());
        return false;
    }

    QDataStream akis(&dosya);
    akis.setVersion(QDataStream::Qt_6_5);
    akis << MAGIC << VERSION;

    // Ogrencileri yaz
    const auto ogrenciler = servis.tumOgrenciler();
    akis << static_cast<quint32>(ogrenciler.size());
    for (const auto& o : ogrenciler) { akis << o; }

    // Dersleri yaz
    const auto dersler = servis.tumDersler();
    akis << static_cast<quint32>(dersler.size());
    for (const auto& d : dersler) { akis << d; }

    // Notlari yaz
    const auto notlar = servis.tumNotlar();
    akis << static_cast<quint32>(notlar.size());
    for (const auto& n : notlar) { akis << n; }

    if (akis.status() != QDataStream::Ok) {
        m_sonHata = "Yazma sirasinda akis hatasi.";
        return false;
    }
    return true;
}

bool PersistenceManager::yukle(const QString& dosyaYolu,
                               OBYSServis& servis) {
    m_sonHata.clear();
    if (!QFileInfo::exists(dosyaYolu)) {
        m_sonHata = QString("Dosya bulunamadi: %1")
```

```
                            .arg(dosyaYolu);
        return false;
    }

    QFile dosya(dosyaYolu);
    if (!dosya.open(QIODevice::ReadOnly)) {
        m_sonHata = QString("Dosya okunamadi: %1")
                            .arg(dosya.errorString());
        return false;
    }

    QDataStream akis(&dosya);
    akis.setVersion(QDataStream::Qt_6_5);

    quint32 magic = 0, version = 0;
    akis >> magic >> version;

    if (magic != MAGIC) {
        m_sonHata = "Gecersiz dosya imzasi.";
        return false;
    }
    if (version > VERSION) {
        m_sonHata = QString("Desteklenmeyen surum: %1")
                            .arg(version);
        return false;
    }

    servis.temizle();

    // Ogrencileri oku
    quint32 n = 0;
    akis >> n;
    for (quint32 i = 0; i < n; ++i) {
        Ogrenci o; akis >> o;
        if (akis.status() != QDataStream::Ok) {
            m_sonHata = "Ogrenci okunurken hata.";
            return false;
        }
        servis.ogrenciEkle(o);
    }

    // Dersleri oku
    akis >> n;
    for (quint32 i = 0; i < n; ++i) {
        Ders d; akis >> d;
        if (akis.status() != QDataStream::Ok) {
            m_sonHata = "Ders okunurken hata.";
            return false;
        }
        servis.dersEkle(d);
    }

    // Notlari oku
    akis >> n;
    for (quint32 i = 0; i < n; ++i) {
        DersKaydi k; akis >> k;
        if (akis.status() != QDataStream::Ok) {
            m_sonHata = "Not okunurken hata.";
            return false;
```

```
            }
            servis.dersKaydiEkle(k);
        }

        return true;
    }
```

Adım 2.4: CMakeLists.txt Güncelleme

`OBYS_SOURCES` listesine yeni dosyayı ekleyin:

```
set(OBYS_SOURCES
    src/main.cpp
    src/persistence_manager.cpp
    src/persistence_manager.h
)
```

Görev 2.1 – Sorular (5 dk)

a) `MAGIC = 0x4F425953` değeri ASCII olarak hangi harflere karşılık gelir? Dosya başlığına neden böyle bir imza koyuyoruz?

_____

b) `akis.setVersion(QDataStream::Qt_6_5)` satırı olmasaydı ne gibi sorunlar çıkabilirdi?

_____

c) `VERSION > VERSION` kontrolü ile `VERSION != VERSION` kontrolü arasındaki fark nedir? İleri uyumluluk açısından hangisi daha doğrudur?

_____

Kontrol Listesi – Aşama 2

- ☐ `include/obys_servis.h` tamamlandı
- ☐ `src/persistence_manager.h` ve `.cpp` tamamlandı
- ☐ `CMakeLists.txt` güncellendi
- ☐ `ornekVerileriYukle()` çağrıldığında 3 öğrenci ve 2 ders oluşuyor
- ☐ Proje derleniyor

4 Aşama 3: Veri Giriş Formları (60 dk)

Ön Bilgi

Qt Designer ile oluşturulan `.ui` dosyaları, form düzenini XML biçiminde saklar ve `CMAKE_AUTOUIC` sayesinde derleme zamanında C++ koduna dönüştürülür. Bu aşamada üç form oluşturacağız: **öğrenci ekleme**, **ders ekleme** ve **not girişi**.

Adım 3.1: Öğrenci Ekleme Formu (.ui)

`ui/ogrenci_ekle_dialog.ui` dosyasını oluşturun. Qt Designer kullanabilir veya doğrudan XML yazabilirsiniz:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>OgrenciEkleDialog</class>
 <widget class="QDialog" name="OgrenciEkleDialog">
  <property name="windowTitle">
   <string>Ogrenci Ekle</string>
  </property>
  <property name="geometry">
   <rect><x>0</x><y>0</y>
         <width>350</width><height>180</height></rect>
  </property>
  <layout class="QFormLayout" name="formLayout">
   <item row="0" column="0">
    <widget class="QLabel" name="numaraLabel">
     <property name="text">
      <string>Ogrenci No:</string>
     </property>
    </widget>
   </item>
   <item row="0" column="1">
    <widget class="QLineEdit" name="numaraEdit"/>
   </item>
   <item row="1" column="0">
    <widget class="QLabel" name="adSoyadLabel">
     <property name="text">
      <string>Ad Soyad:</string>
     </property>
    </widget>
   </item>
   <item row="1" column="1">
    <widget class="QLineEdit" name="adSoyadEdit"/>
   </item>
   <item row="2" column="0" colspan="2">
    <widget class="QDialogButtonBox"
            name="buttonBox">
     <property name="standardButtons">
      <set>QDialogButtonBox::Ok
           |QDialogButtonBox::Cancel</set>
     </property>
    </widget>
   </item>
  </layout>
 </widget>
</ui>
```

Adım 3.2: Ders Ekleme Formu (.ui)

`ui/ders_ekle_dialog.ui` dosyasını oluşturun:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>DersEkleDialog</class>
 <widget class="QDialog" name="DersEkleDialog">
  <property name="windowTitle">
   <string>Ders Ekle</string>
  </property>
  <layout class="QFormLayout" name="formLayout">
   <item row="0" column="0">
    <widget class="QLabel" name="kodLabel">
     <property name="text">
      <string>Ders Kodu:</string></property>
    </widget>
   </item>
   <item row="0" column="1">
    <widget class="QLineEdit" name="kodEdit"/>
   </item>
   <item row="1" column="0">
    <widget class="QLabel" name="adLabel">
     <property name="text">
      <string>Ders Adi:</string></property>
    </widget>
   </item>
   <item row="1" column="1">
    <widget class="QLineEdit" name="adEdit"/>
   </item>
   <item row="2" column="0">
    <widget class="QLabel" name="krediLabel">
     <property name="text">
      <string>Kredi:</string></property>
    </widget>
   </item>
   <item row="2" column="1">
    <widget class="QSpinBox" name="krediSpin">
     <property name="minimum"><number>1</number>
     </property>
     <property name="maximum"><number>10</number>
     </property>
     <property name="value"><number>3</number>
     </property>
    </widget>
   </item>
   <item row="3" column="0" colspan="2">
    <widget class="QDialogButtonBox"
            name="buttonBox">
     <property name="standardButtons">
      <set>QDialogButtonBox::Ok
           |QDialogButtonBox::Cancel</set>
     </property>
    </widget>
   </item>
  </layout>
 </widget>
</ui>
```

Adım 3.3: Not Giriş Diyalogu

Not giriş formunu C++ ile oluşturacağız çünkü dinamik davranış (`std::variant` türüne göre kontrol etkinleştirme/devre dışı bırakma) gerektirir.

`src/not_giris_dialog.h`:

```
#ifndef NOT_GIRIS_DIALOG_H
#define NOT_GIRIS_DIALOG_H

#include "obys_domain.h"
#include <QDialog>
#include <QList>
#include <QPair>

class QComboBox;
class QDoubleSpinBox;
class QLabel;
class QDialogButtonBox;

class NotGirisDialog : public QDialog {
    Q_OBJECT
public:
    explicit NotGirisDialog(QWidget* parent = nullptr);

    NotDTO notDTO() const;

    void ogrencileriYukle(
        const QList<QPair<int, QString>>& liste);
    void dersleriYukle(
        const QList<QPair<int, QString>>& liste);

private slots:
    void notDegistiginde(double yeniDeger);
    void notTuruDegistiginde(int indeks);

private:
    static QString sayisalDanHarfe(double puan);

    QComboBox*        m_ogrenciSecici{nullptr};
    QComboBox*        m_dersSecici{nullptr};
    QDoubleSpinBox*   m_notDegeri{nullptr};
    QComboBox*        m_notTuruSecici{nullptr};
    QLabel*           m_harfOnizleme{nullptr};
    QDialogButtonBox* m_butonlar{nullptr};
};

#endif // NOT_GIRIS_DIALOG_H
```

`src/not_giris_dialog.cpp`:

```
#include "not_giris_dialog.h"

#include <QComboBox>
#include <QDialogButtonBox>
#include <QDoubleSpinBox>
#include <QFormLayout>
#include <QLabel>
#include <QVBoxLayout>

NotGirisDialog::NotGirisDialog(QWidget* parent)
    : QDialog(parent) {
```

```
    setWindowTitle("Ders Notu Girişi");
    setModal(true);

    m_ogrenciSecici = new QComboBox(this);
    m_dersSecici    = new QComboBox(this);

    m_notTuruSecici = new QComboBox(this);
    m_notTuruSecici->addItem("Sayısal", 0);
    m_notTuruSecici->addItem("Harf",    1);
    m_notTuruSecici->addItem("Muaf",    2);

    m_notDegeri = new QDoubleSpinBox(this);
    m_notDegeri->setRange(0.0, 100.0);
    m_notDegeri->setDecimals(2);
    m_notDegeri->setSingleStep(0.5);

    m_harfOnizleme = new QLabel(
        "Harf Önizleme: -", this);

    auto* form = new QFormLayout;
    form->addRow("Öğrenci:", m_ogrenciSecici);
    form->addRow("Ders:",    m_dersSecici);
    form->addRow("Not Türü:", m_notTuruSecici);
    form->addRow("Not Değeri:", m_notDegeri);
    form->addRow(m_harfOnizleme);

    m_butonlar = new QDialogButtonBox(
        QDialogButtonBox::Ok
        | QDialogButtonBox::Cancel, this);

    auto* ana = new QVBoxLayout(this);
    ana->addLayout(form);
    ana->addWidget(m_butonlar);

    connect(m_butonlar,
            &QDialogButtonBox::accepted,
            this, &QDialog::accept);
    connect(m_butonlar,
            &QDialogButtonBox::rejected,
            this, &QDialog::reject);

    connect(m_notDegeri,
            qOverload<double>(
                &QDoubleSpinBox::valueChanged),
            this,
            &NotGirisDialog::notDegistiginde);
    connect(m_notTuruSecici,
            qOverload<int>(
                &QComboBox::currentIndexChanged),
            this,
            &NotGirisDialog::notTuruDegistiginde);

    notTuruDegistiginde(0);
}

void NotGirisDialog::ogrencileriYukle(
    const QList<QPair<int, QString>>& liste) {
    m_ogrenciSecici->clear();
    for (const auto& [id, ad] : liste) {
```

```
        m_ogrenciSecici->addItem(ad, id);
    }
}

void NotGirisDialog::dersleriYukle(
    const QList<QPair<int, QString>>& liste) {
    m_dersSecici->clear();
    for (const auto& [id, ad] : liste) {
        m_dersSecici->addItem(ad, id);
    }
}

NotDTO NotGirisDialog::notDTO() const {
    NotDTO dto;
    dto.ogrenciId =
        m_ogrenciSecici->currentData().toInt();
    dto.dersId =
        m_dersSecici->currentData().toInt();

    switch (m_notTuruSecici->currentIndex()) {
    case 0:
        dto.notu = SayisalNot{m_notDegeri->value()};
        break;
    case 1: {
        HarfNotu h;
        h.harf = sayisalDanHarfe(
                     m_notDegeri->value());
        dto.notu = h;
        break;
    }
    case 2:
        dto.notu = MuafNot{};
        break;
    }
    return dto;
}

void NotGirisDialog::notDegistiginde(
    double yeniDeger) {
    m_harfOnizleme->setText(
        QString("Harf: %1")
            .arg(sayisalDanHarfe(yeniDeger)));
}

void NotGirisDialog::notTuruDegistiginde(
    int indeks) {
    const bool aktif = (indeks != 2);
    m_notDegeri->setEnabled(aktif);
    m_harfOnizleme->setVisible(aktif);
    if (!aktif) {
        m_harfOnizleme->setText("Harf: MUAF");
    } else {
        notDegistiginde(m_notDegeri->value());
    }
}

QString NotGirisDialog::sayisalDanHarfe(
    double puan) {
    if (puan >= 90.0) return "AA";
```

```
    if (puan >= 85.0) return "BA";
    if (puan >= 80.0) return "BB";
    if (puan >= 75.0) return "CB";
    if (puan >= 70.0) return "CC";
    if (puan >= 65.0) return "DC";
    if (puan >= 60.0) return "DD";
    if (puan >= 50.0) return "FD";
    return "FF";
}
```

Adım 3.4: CMakeLists.txt Güncelleme

`OBYS_SOURCES` listesini genişletin:

```
set(OBYS_SOURCES
    src/main.cpp
    src/persistence_manager.cpp
    src/persistence_manager.h
    src/not_giris_dialog.cpp
    src/not_giris_dialog.h
    ui/ogrenci_ekle_dialog.ui
    ui/ders_ekle_dialog.ui
)
```

Görev 3.1 – Sorular (5 dk)

a) `.ui` dosyaları derleme zamanında neye dönüştürülür? `CMAKE_AUTOUIC` ne iş yapar?

_____

b) `qOverload<double>(&QDoubleSpinBox::valueChanged)` neden gereklidir? Doğrudan sinyal bağlasaydık ne olurdu?

_____

c) `setModal(true)` çağrısının etkisi nedir? Bunu yazmasaydık kullanıcı deneyimi nasıl değişirdi?

_____

Kontrol Listesi – Aşama 3

- ☐ `ui/ogrenci_ekle_dialog.ui` oluşturuldu
- ☐ `ui/ders_ekle_dialog.ui` oluşturuldu
- ☐ `src/not_giris_dialog.h` ve `.cpp` tamamlandı
- ☐ Not türü “Muaf” seçildiğinde not değeri alanı devre dışı kalıyor
- ☐ Harf önizlemesi not değeri değiştikçe güncelleniyor
- ☐ Proje derleniyor

5 Aşama 4: Ana Pencere (75 dk)

Ön Bilgi

Ana pencere uygulamanın kalbidir. `QMainWindow` sınıfından türeyecek; menü çubuğu, durum çubuğu, sekmeli merkez widget ve dosya yönetimi içerecektir. `NotlarModel` adında özel bir `QAbstractTableModel` ile not kayıtlarını tablo halinde göstereceğiz.

Adım 4.1: NotlarModel – Özel Tablo Modeli

`src/notlar_model.h`:

```
#ifndef NOTLAR_MODEL_H
#define NOTLAR_MODEL_H

#include "obys_servis.h"
#include <QAbstractTableModel>

class NotlarModel : public QAbstractTableModel {
    Q_OBJECT
public:
    explicit NotlarModel(OBYSServis* servis,
                         QObject* parent = nullptr);

    void yenile();

    int rowCount(const QModelIndex& parent
        = QModelIndex()) const override;
    int columnCount(const QModelIndex& parent
        = QModelIndex()) const override;
    QVariant data(const QModelIndex& idx,
        int role = Qt::DisplayRole) const override;
    QVariant headerData(int section,
        Qt::Orientation orient,
        int role = Qt::DisplayRole) const override;

private:
    OBYSServis* m_servis{nullptr};
};

#endif // NOTLAR_MODEL_H
```

`src/notlar_model.cpp`:

```
#include "notlar_model.h"
#include "obys_domain.h"
#include <variant>

namespace {
QString sayisalHarf(double puan) {
    if (puan >= 90) return "AA";
    if (puan >= 85) return "BA";
    if (puan >= 80) return "BB";
    if (puan >= 75) return "CB";
    if (puan >= 65) return "CC";
    if (puan >= 58) return "DC";
    if (puan >= 50) return "DD";
    return "FF";
```

```
}

QString ogrenciAdi(int id,
                   const OBYSServis& servis) {
    for (const auto& o : servis.tumOgrenciler()) {
        if (o.id == id) return o.adSoyad;
    }
    return "?";
}
} // namespace

NotlarModel::NotlarModel(OBYSServis* servis,
                         QObject* parent)
    : QAbstractTableModel(parent)
    , m_servis(servis) {}

void NotlarModel::yenile() {
    beginResetModel();
    endResetModel();
}

int NotlarModel::rowCount(
    const QModelIndex& parent) const {
    if (parent.isValid() || !m_servis) return 0;
    return m_servis->tumNotlar().size();
}

int NotlarModel::columnCount(
    const QModelIndex& parent) const {
    if (parent.isValid()) return 0;
    return 4; // #, Ogrenci, Not, Harf
}

QVariant NotlarModel::data(const QModelIndex& idx,
                           int role) const {
    if (!idx.isValid() || !m_servis) return {};
    if (role != Qt::DisplayRole) return {};

    const auto notlar = m_servis->tumNotlar();
    const int satir = idx.row();
    if (satir < 0 || satir >= notlar.size())
        return {};

    const DersKaydi& k = notlar.at(satir);

    switch (idx.column()) {
    case 0: return satir + 1;
    case 1: return ogrenciAdi(k.ogrenciId,
                              *m_servis);
    case 2: {
        QString s;
        std::visit([&s](const auto& d) {
            using T = std::decay_t<decltype(d)>;
            if constexpr (
                std::is_same_v<T, SayisalNot>)
                s = QString::number(d.deger, 'f', 1);
            else if constexpr (
                std::is_same_v<T, HarfNotu>)
                s = d.harf;
```

```
            else if constexpr (
                std::is_same_v<T, MuafNot>)
                s = "MUAF";
        }, k.notu);
        return s;
    }
    case 3: {
        QString h;
        std::visit([&h](const auto& d) {
            using T = std::decay_t<decltype(d)>;
            if constexpr (
                std::is_same_v<T, SayisalNot>)
                h = sayisalHarf(d.deger);
            else if constexpr (
                std::is_same_v<T, HarfNotu>)
                h = d.harf;
            else if constexpr (
                std::is_same_v<T, MuafNot>)
                h = "-";
        }, k.notu);
        return h;
    }
    default: return {};
    }
}

QVariant NotlarModel::headerData(
    int section, Qt::Orientation orient,
    int role) const {
    if (role != Qt::DisplayRole) return {};
    if (orient == Qt::Horizontal) {
        switch (section) {
        case 0: return "#";
        case 1: return "Öğrenci";
        case 2: return "Not";
        case 3: return "Harf";
        }
    }
    return section + 1;
}
```

Adım 4.2: MainWindow Başlık Dosyası

`src/main_window.h`:

```
#ifndef MAIN_WINDOW_H
#define MAIN_WINDOW_H

#include "obys_servis.h"
#include "persistence_manager.h"

#include <QMainWindow>
#include <QStringList>

class QAction;
class QMenu;
class QTabWidget;
class QTableView;
class QPushButton;
```

```
class QTextDocument;
class NotlarModel;

class MainWindow : public QMainWindow {
    Q_OBJECT

public:
    explicit MainWindow(QWidget* parent = nullptr);

protected:
    void closeEvent(QCloseEvent* olay) override;

private slots:
    void dosyaYeni();
    void dosyaAc();
    bool dosyaKaydet();
    bool dosyaFarkliKaydet();

    void ogrenciEkleDialogAc();
    void dersEkleDialogAc();
    void notEkleDialogAc();

private:
    void menuleriKur();
    void merkeziWidgetKur();
    QWidget* notlarSekmesiKur();

    void basligiGuncelle();
    bool kaydetmeSorusu();
    void dirtyYap(bool d = true);

    void ayarlariYukle();
    void ayarlariKaydet();

    // Son acilanlar
    void sonAcilanlariYukle();
    void sonAcilanlariGuncelle(const QString& yol);
    void sonAcilanlarMenusunuYenile();
    void dosyaYoluAc(const QString& yol);

    OBYSServis         m_servis;
    PersistenceManager m_persist;

    QString     m_aktifDosya;
    bool        m_dirty{false};

    QTabWidget*  m_sekmeler{nullptr};
    QTableView*  m_notTablosu{nullptr};
    NotlarModel* m_notlarModel{nullptr};
    QPushButton* m_notEkleButonu{nullptr};

    // Menuler
    QMenu*      m_sonAcilanlarMenu{nullptr};
    QStringList m_sonAcilanlar;
    static constexpr int kMaxSonAcilan = 5;
};

#endif // MAIN_WINDOW_H
```

Adım 4.3: MainWindow Uygulama – Parça 1: Yapıcı ve Merkez Widget

`src/main_window.cpp` dosyasını oluşturun (uzun dosya, parça parça ekliyoruz):

```cpp
#include "main_window.h"
#include "not_giris_dialog.h"
#include "notlar_model.h"
#include "obys_domain.h"
#include "ui_ogrenci_ekle_dialog.h"
#include "ui_ders_ekle_dialog.h"

#include <QAction>
#include <QCloseEvent>
#include <QFileDialog>
#include <QFileInfo>
#include <QHeaderView>
#include <QInputDialog>
#include <QMenu>
#include <QMenuBar>
#include <QMessageBox>
#include <QPushButton>
#include <QSettings>
#include <QStatusBar>
#include <QTableView>
#include <QTabWidget>
#include <QVBoxLayout>

MainWindow::MainWindow(QWidget* parent)
    : QMainWindow(parent) {
    merkeziWidgetKur();
    menuleriKur();

    m_servis.ornekVerileriYukle();

    // Ornek notlar
    m_servis.dersKaydiEkle(
        {1, 10, SayisalNot{92.0}});
    m_servis.dersKaydiEkle(
        {2, 10, SayisalNot{68.0}});
    m_servis.dersKaydiEkle(
        {3, 10, SayisalNot{55.0}});
    m_servis.dersKaydiEkle(
        {1, 11, SayisalNot{85.0}});

    m_notlarModel->yenile();
    m_aktifDosya.clear();
    m_dirty = false;
    basligiGuncelle();

    ayarlariYukle();
    sonAcilanlariYukle();
    sonAcilanlarMenusunuYenile();

    statusBar()->showMessage("Hazır", 2000);
    if (size().isEmpty()) resize(900, 600);
}
```

Adım 4.4: Merkezi Widget ve Not Sekmesi

Aynı dosyada devam:

```
QWidget* MainWindow::notlarSekmesiKur() {
    auto* sayfa    = new QWidget(this);
    auto* yerlesim = new QVBoxLayout(sayfa);

    m_notTablosu  = new QTableView(sayfa);
    m_notlarModel = new NotlarModel(
                        &m_servis, this);
    m_notTablosu->setModel(m_notlarModel);
    m_notTablosu->horizontalHeader()
                ->setStretchLastSection(true);
    m_notTablosu->setAlternatingRowColors(true);
    m_notTablosu->setSelectionBehavior(
        QAbstractItemView::SelectRows);
    m_notTablosu->setEditTriggers(
        QAbstractItemView::NoEditTriggers);

    m_notEkleButonu =
        new QPushButton("Not Ekle...", sayfa);

    yerlesim->addWidget(m_notTablosu);
    yerlesim->addWidget(m_notEkleButonu);

    connect(m_notEkleButonu,
            &QPushButton::clicked,
            this,
            &MainWindow::notEkleDialogAc);

    return sayfa;
}

void MainWindow::merkeziWidgetKur() {
    m_sekmeler = new QTabWidget(this);
    m_sekmeler->addTab(
        notlarSekmesiKur(), "Notlar");
    // Grafik ve trend sekmeleri
    // Asama 6'da eklenecek
    setCentralWidget(m_sekmeler);
}
```

Adım 4.5: Menü Kurulumu

```
void MainWindow::menuleriKur() {
    // Dosya menusu
    auto* dosyaMenu =
        menuBar()->addMenu("&Dosya");

    auto* yeniAct =
        dosyaMenu->addAction("&Yeni");
    yeniAct->setShortcut(QKeySequence::New);
    connect(yeniAct, &QAction::triggered,
            this, &MainWindow::dosyaYeni);

    auto* acAct =
        dosyaMenu->addAction("&Aç...");
    acAct->setShortcut(QKeySequence::Open);
```

```
    connect(acAct, &QAction::triggered,
            this, &MainWindow::dosyaAc);

    m_sonAcilanlarMenu =
        dosyaMenu->addMenu("Son Açılanlar");

    dosyaMenu->addSeparator();

    auto* kaydetAct =
        dosyaMenu->addAction("&Kaydet");
    kaydetAct->setShortcut(QKeySequence::Save);
    connect(kaydetAct, &QAction::triggered,
            this, [this]{ dosyaKaydet(); });

    auto* farkliAct = dosyaMenu->addAction(
        "&Farklı Kaydet...");
    farkliAct->setShortcut(QKeySequence::SaveAs);
    connect(farkliAct, &QAction::triggered,
            this, [this]{ dosyaFarkliKaydet(); });

    dosyaMenu->addSeparator();

    auto* cikisAct = dosyaMenu->addAction(
        "Çıkış");
    cikisAct->setShortcut(QKeySequence::Quit);
    connect(cikisAct, &QAction::triggered,
            this, &MainWindow::close);

    // Veri menusu
    auto* veriMenu = menuBar()->addMenu(
        "&Veri");

    auto* ogrEkle = veriMenu->addAction(
        "Öğrenci Ekle...");
    connect(ogrEkle, &QAction::triggered,
            this,
            &MainWindow::ogrenciEkleDialogAc);

    auto* dersEkle = veriMenu->addAction(
        "Ders Ekle...");
    connect(dersEkle, &QAction::triggered,
            this,
            &MainWindow::dersEkleDialogAc);

    veriMenu->addSeparator();

    auto* notEkleAct = veriMenu->addAction(
        "Not Ekle...");
    notEkleAct->setShortcut(
        QKeySequence(Qt::CTRL | Qt::SHIFT
                     | Qt::Key_N));
    connect(notEkleAct, &QAction::triggered,
            this,
            &MainWindow::notEkleDialogAc);
}
```

Adım 4.6: Dosya İşlemleri ve Yardımcı Metotlar

```
void MainWindow::basligiGuncelle() {
    const QString ad = m_aktifDosya.isEmpty()
        ? "İsimsiz"
        : QFileInfo(m_aktifDosya).fileName();
    const QString yildiz = m_dirty ? " *" : "";
    setWindowTitle(
        QString("OBYS - %1%2").arg(ad, yildiz));
}

void MainWindow::dirtyYap(bool d) {
    m_dirty = d;
    basligiGuncelle();
}

bool MainWindow::kaydetmeSorusu() {
    if (!m_dirty) return true;
    const auto s = QMessageBox::question(
        this, "Kaydedilmemiş Değişiklikler",
        "Kaydedilsin mi?",
        QMessageBox::Save | QMessageBox::Discard
        | QMessageBox::Cancel,
        QMessageBox::Save);
    if (s == QMessageBox::Save)
        return dosyaKaydet();
    if (s == QMessageBox::Cancel)
        return false;
    return true;
}

void MainWindow::dosyaYeni() {
    if (!kaydetmeSorusu()) return;
    m_servis.ornekVerileriYukle();
    m_notlarModel->yenile();
    m_aktifDosya.clear();
    m_dirty = false;
    basligiGuncelle();
}

void MainWindow::dosyaAc() {
    if (!kaydetmeSorusu()) return;
    const QString yol =
        QFileDialog::getOpenFileName(
            this, "OBYS Dosyası Aç", "",
            "OBYS (*.obys);;Tumu (*)");
    if (yol.isEmpty()) return;
    dosyaYoluAc(yol);
}

void MainWindow::dosyaYoluAc(
    const QString& yol) {
    if (!m_persist.yukle(yol, m_servis)) {
        QMessageBox::warning(this, "Hata",
            m_persist.sonHata());
        return;
    }
    m_notlarModel->yenile();
    m_aktifDosya = yol;
    m_dirty = false;
    basligiGuncelle();
```

```
    sonAcilanlariGuncelle(yol);
}

bool MainWindow::dosyaKaydet() {
    if (m_aktifDosya.isEmpty())
        return dosyaFarkliKaydet();
    if (!m_persist.kaydet(
            m_aktifDosya, m_servis)) {
        QMessageBox::warning(this, "Hata",
            m_persist.sonHata());
        return false;
    }
    m_dirty = false;
    basligiGuncelle();
    sonAcilanlariGuncelle(m_aktifDosya);
    return true;
}

bool MainWindow::dosyaFarkliKaydet() {
    QString yol =
        QFileDialog::getSaveFileName(
            this, "Farklı Kaydet",
            "oturum.obys",
            "OBYS (*.obys)");
    if (yol.isEmpty()) return false;
    if (!yol.endsWith(".obys",
            Qt::CaseInsensitive))
        yol += ".obys";
    if (!m_persist.kaydet(yol, m_servis)) {
        QMessageBox::warning(this, "Hata",
            m_persist.sonHata());
        return false;
    }
    m_aktifDosya = yol;
    m_dirty = false;
    basligiGuncelle();
    sonAcilanlariGuncelle(yol);
    return true;
}
```

Adım 4.7: Öğrenci ve Ders Ekleme (.ui Kullanımı)

```
void MainWindow::ogrenciEkleDialogAc() {
    QDialog dlg(this);
    Ui::OgrenciEkleDialog ui;
    ui.setupUi(&dlg);

    connect(ui.buttonBox,
            &QDialogButtonBox::accepted,
            &dlg, &QDialog::accept);
    connect(ui.buttonBox,
            &QDialogButtonBox::rejected,
            &dlg, &QDialog::reject);

    if (dlg.exec() != QDialog::Accepted) return;

    const QString numara =
        ui.numaraEdit->text().trimmed();
```

```
    const QString adSoyad =
        ui.adSoyadEdit->text().trimmed();
    if (numara.isEmpty() || adSoyad.isEmpty()) {
        QMessageBox::warning(this, "Uyarı",
            "Tüm alanları doldurun.");
        return;
    }

    // Basit ID atama
    int yeniId = 1;
    for (const auto& o :
         m_servis.tumOgrenciler()) {
        if (o.id >= yeniId) yeniId = o.id + 1;
    }

    m_servis.ogrenciEkle(
        {yeniId, numara, adSoyad});
    dirtyYap(true);
    statusBar()->showMessage(
        "Öğrenci eklendi: " + adSoyad, 2000);
}

void MainWindow::dersEkleDialogAc() {
    QDialog dlg(this);
    Ui::DersEkleDialog ui;
    ui.setupUi(&dlg);

    connect(ui.buttonBox,
            &QDialogButtonBox::accepted,
            &dlg, &QDialog::accept);
    connect(ui.buttonBox,
            &QDialogButtonBox::rejected,
            &dlg, &QDialog::reject);

    if (dlg.exec() != QDialog::Accepted) return;

    const QString kod =
        ui.kodEdit->text().trimmed();
    const QString ad =
        ui.adEdit->text().trimmed();
    const int kredi = ui.krediSpin->value();
    if (kod.isEmpty() || ad.isEmpty()) {
        QMessageBox::warning(this, "Uyarı",
            "Tüm alanları doldurun.");
        return;
    }

    int yeniId = 1;
    for (const auto& d : m_servis.tumDersler()) {
        if (d.id >= yeniId) yeniId = d.id + 1;
    }

    m_servis.dersEkle({yeniId, kod, ad, kredi});
    dirtyYap(true);
    statusBar()->showMessage(
        "Ders eklendi: " + ad, 2000);
}
```

Adım 4.8: Not Ekleme ve Diğer Yardımcılar

```
void MainWindow::notEkleDialogAc() {
    NotGirisDialog dlg(this);

    QList<QPair<int, QString>> ogrenciler;
    for (const auto& o :
         m_servis.tumOgrenciler()) {
        ogrenciler.append({o.id, o.adSoyad});
    }
    QList<QPair<int, QString>> dersler;
    for (const auto& d : m_servis.tumDersler()) {
        dersler.append(
            {d.id,
             QString("%1 %2").arg(d.kod, d.ad)});
    }
    dlg.ogrencileriYukle(ogrenciler);
    dlg.dersleriYukle(dersler);

    if (dlg.exec() == QDialog::Accepted) {
        m_servis.notEkle(dlg.notDTO());
        m_notlarModel->yenile();
        dirtyYap(true);
        statusBar()->showMessage(
            "Not eklendi", 2000);
    }
}

// QSettings
void MainWindow::ayarlariYukle() {
    QSettings s;
    const QByteArray g =
        s.value("pencere/geometri").toByteArray();
    if (!g.isEmpty()) restoreGeometry(g);
    const QByteArray d =
        s.value("pencere/durum").toByteArray();
    if (!d.isEmpty()) restoreState(d);
}

void MainWindow::ayarlariKaydet() {
    QSettings s;
    s.setValue("pencere/geometri",
               saveGeometry());
    s.setValue("pencere/durum",
               saveState());
}

// Son acilanlar
void MainWindow::sonAcilanlariYukle() {
    QSettings s;
    m_sonAcilanlar =
        s.value("dosya/sonAcilanlar")
         .toStringList();
    QStringList temiz;
    for (const auto& y : m_sonAcilanlar) {
        if (QFileInfo::exists(y))
            temiz.append(y);
    }
    m_sonAcilanlar = temiz;
```

```
}

void MainWindow::sonAcilanlariGuncelle(
    const QString& yol) {
    m_sonAcilanlar.removeAll(yol);
    m_sonAcilanlar.prepend(yol);
    while (m_sonAcilanlar.size() > kMaxSonAcilan)
        m_sonAcilanlar.removeLast();
    QSettings s;
    s.setValue("dosya/sonAcilanlar",
               m_sonAcilanlar);
    sonAcilanlarMenusunuYenile();
}

void MainWindow::sonAcilanlarMenusunuYenile() {
    if (!m_sonAcilanlarMenu) return;
    m_sonAcilanlarMenu->clear();
    if (m_sonAcilanlar.isEmpty()) {
        auto* bos = m_sonAcilanlarMenu->addAction(
            "(boş)");
        bos->setEnabled(false);
        return;
    }
    int sira = 1;
    for (const auto& yol : m_sonAcilanlar) {
        auto* act =
            m_sonAcilanlarMenu->addAction(
                QString("&%1 %2").arg(sira)
                    .arg(QFileInfo(yol)
                        .fileName()));
        connect(act, &QAction::triggered,
                this, [this, yol]{
            if (kaydetmeSorusu())
                dosyaYoluAc(yol);
        });
        ++sira;
    }
}

void MainWindow::closeEvent(
    QCloseEvent* olay) {
    if (!kaydetmeSorusu()) {
        olay->ignore();
        return;
    }
    ayarlariKaydet();
    olay->accept();
}
```

Adım 4.9: main.cpp Güncelleme

`src/main.cpp` dosyasını güncelleyin:

```
#include "main_window.h"
#include <QApplication>

int main(int argc, char* argv[]) {
    QApplication uygulama(argc, argv);

```

```
    QCoreApplication::setOrganizationName("KTU");
    QCoreApplication::setApplicationName(
        "OBYS Tutorial");
    QCoreApplication::setApplicationVersion(
        "1.0.0");

    MainWindow pencere;
    pencere.show();
    return uygulama.exec();
}
```

Adım 4.10: CMakeLists.txt Güncelleme

```
set(OBYS_SOURCES
    src/main.cpp
    src/main_window.cpp          src/main_window.h
    src/persistence_manager.cpp  src/persistence_manager.h
    src/not_giris_dialog.cpp     src/not_giris_dialog.h
    src/notlar_model.cpp         src/notlar_model.h
    ui/ogrenci_ekle_dialog.ui
    ui/ders_ekle_dialog.ui
)
```

Görev 4.1 – Sorular (10 dk)

a) `QAbstractTableModel`'den türetilen `NotlarModel` sınıfında en az hangi 4 metot override edilmelidir?

_____

b) `beginResetModel()` / `endResetModel()` çifti çağrılmazsa ne olur?

_____

c) `ui/ogrenci_ekle_dialog.ui` dosyasını kullanırken `Ui::OgrenciEkleDialog` sınıfını `include` etmek yerine hangi başlık dosyasını ekliyoruz? Bu dosya nerede oluşturuluyor?

_____

d) "Son açılanlar" menüsü hangi mekanizma ile kalıcı hale getiriliyor? Veriler nerede saklanıyor?

_____

Kontrol Listesi – Aşama 4

- ☐ Ana pencere Dosya ve “Veri menüleriyle açılıyor
- ☐ “Notlar” sekmesinde tablo görünümü mevcut
- ☐ Örnek veriler tabloda listeleniyor
- ☐ ““Veri → Öğrenci Ekle” ile öğrenci eklenebiliyor (.ui formu)
- ☐ ““Veri → Ders Ekle” ile ders eklenebiliyor (.ui formu)
- ☐ “Not Ekle” diyalogu açılıyor ve not kaydediliyor
- ☐ Dosya → Kaydet / Farklı Kaydet çalışıyor (.obys dosyası)
- ☐ Dosya → Aç ile kayıtlı dosya yüklenebiliyor
- ☐ Pencere başlığında dosya adı ve “*” işareti görünüyor
- ☐ Kapanışta “Kaydedilmemiş değişiklikler” soruluyor

6 Aşama 5: DTO, JSON Dışa Aktarım ve Ayarlar Penceresi (45 dk)

Ön Bilgi

DTO (Data Transfer Object) deseni, veri taşıma nesneleri ile iş mantığı nesnelerini ayırır. Bu aşamada JSON dışa aktarım ve kullanıcının tercihlerini düzenleyebileceği bir **Ayarlar Penceresi** ekleyeceğiz. Ayarlar penceresi `.ui` dosyası ile tasarlanacaktır.

Adım 5.1: JsonExporter Sınıfı

`src/json_exporter.h`:

```
#ifndef JSON_EXPORTER_H
#define JSON_EXPORTER_H

#include "obys_servis.h"
#include <QJsonObject>
#include <QString>

class JsonExporter {
public:
    bool disaAktar(const QString& yol,
                   const OBYSServis& servis);
    QString sonHata() const
        { return m_sonHata; }

private:
    QJsonObject ogrenciyeJson(
        const Ogrenci& o) const;
    QJsonObject dersyeJson(
        const Ders& d) const;
    QJsonObject notKaydiJson(
        const DersKaydi& k) const;
    QJsonObject notTuruyuJson(
        const NotTuru& n) const;

    QString m_sonHata;
};

#endif // JSON_EXPORTER_H
```

`src/json_exporter.cpp`:

```
#include "json_exporter.h"

#include <QDateTime>
#include <QJsonArray>
#include <QJsonDocument>
#include <QSaveFile>
#include <variant>

// overloaded helper (C++17)
template <class... Ts>
struct overloaded : Ts... {
    using Ts::operator()...;
};
template <class... Ts>
overloaded(Ts...) -> overloaded<Ts...>;
```

```

QJsonObject JsonExporter::notTuruyuJson(
    const NotTuru& n) const {
    QJsonObject j;
    std::visit(overloaded{
        [&j](const SayisalNot& s) {
            j["tur"]   = "sayisal";
            j["deger"] = s.deger;
        },
        [&j](const HarfNotu& h) {
            j["tur"]  = "harf";
            j["harf"] = h.harf;
        },
        [&j](const MuafNot&) {
            j["tur"] = "muaf";
        }
    }, n);
    return j;
}

QJsonObject JsonExporter::ogrenciyeJson(
    const Ogrenci& o) const {
    return {{"id", o.id},
            {"numara", o.numara},
            {"adSoyad", o.adSoyad}};
}

QJsonObject JsonExporter::dersyeJson(
    const Ders& d) const {
    return {{"id", d.id}, {"kod", d.kod},
            {"ad", d.ad}, {"kredi", d.kredi}};
}

QJsonObject JsonExporter::notKaydiJson(
    const DersKaydi& k) const {
    return {{"ogrenciId", k.ogrenciId},
            {"dersId", k.dersId},
            {"not", notTuruyuJson(k.notu)}};
}

bool JsonExporter::disaAktar(
    const QString& yol,
    const OBYSServis& servis) {
    m_sonHata.clear();

    QJsonObject kok;
    kok["surum"] = "1.0.0";
    kok["olusturulmaZamani"] =
        QDateTime::currentDateTime()
            .toString(Qt::ISODate);

    QJsonArray ogrArr;
    for (const auto& o :
         servis.tumOgrenciler())
        ogrArr.append(ogrenciyeJson(o));
    kok["ogrenciler"] = ogrArr;

    QJsonArray drsArr;
    for (const auto& d : servis.tumDersler())
```

```
        drsArr.append(dersyeJson(d));
    kok["dersler"] = drsArr;

    QJsonArray notArr;
    for (const auto& k : servis.tumNotlar())
        notArr.append(notKaydiJson(k));
    kok["notlar"] = notArr;

    QSaveFile dosya(yol);
    if (!dosya.open(QIODevice::WriteOnly)) {
        m_sonHata = dosya.errorString();
        return false;
    }
    const QByteArray veri =
        QJsonDocument(kok).toJson(
            QJsonDocument::Indented);
    dosya.write(veri);
    if (!dosya.commit()) {
        m_sonHata = dosya.errorString();
        return false;
    }
    return true;
}
```

Adım 5.2: Ayarlar Penceresi (.ui Dosyası)

`ui/ayarlar_dialog.ui` dosyasını oluşturun:

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>AyarlarDialog</class>
 <widget class="QDialog" name="AyarlarDialog">
  <property name="windowTitle">
   <string>Ayarlar</string>
  </property>
  <property name="geometry">
   <rect><x>0</x><y>0</y>
         <width>400</width><height>300</height>
   </rect>
  </property>
  <layout class="QVBoxLayout" name="anaLayout">
   <item>
    <widget class="QGroupBox" name="temaGroup">
     <property name="title">
      <string>Tema</string>
     </property>
     <layout class="QVBoxLayout">
      <item>
       <widget class="QRadioButton"
               name="aydinlikRadio">
        <property name="text">
         <string>Aydinlik Tema</string>
        </property>
        <property name="checked">
         <bool>true</bool>
        </property>
       </widget>
      </item>
      <item>
```

```
32          <widget class="QRadioButton"
33                  name="karanlikRadio">
34           <property name="text">
35            <string>Karanlik Tema</string>
36           </property>
37          </widget>
38         </item>
39        </layout>
40       </widget>
41      </item>
42      <item>
43       <widget class="QGroupBox"
44               name="kayitGroup">
45        <property name="title">
46         <string>Dosya Ayarlari</string>
47        </property>
48        <layout class="QFormLayout">
49         <item row="0" column="0">
50          <widget class="QLabel">
51           <property name="text">
52            <string>Otomatik Kaydetme (dk):</string>
53           </property>
54          </widget>
55         </item>
56         <item row="0" column="1">
57          <widget class="QSpinBox"
58                  name="otomatikKaydetSpin">
59           <property name="minimum">
60            <number>0</number></property>
61           <property name="maximum">
62            <number>60</number></property>
63           <property name="value">
64            <number>5</number></property>
65           <property name="specialValueText">
66            <string>Kapali</string></property>
67          </widget>
68         </item>
69         <item row="1" column="0">
70          <widget class="QLabel">
71           <property name="text">
72            <string>Son Acilanlar Sayisi:</string>
73           </property>
74          </widget>
75         </item>
76         <item row="1" column="1">
77          <widget class="QSpinBox"
78                  name="sonAcilanSayiSpin">
79           <property name="minimum">
80            <number>1</number></property>
81           <property name="maximum">
82            <number>20</number></property>
83           <property name="value">
84            <number>5</number></property>
85          </widget>
86         </item>
87        </layout>
88       </widget>
89      </item>
90      <item>
```

```
      <widget class="QDialogButtonBox"
              name="buttonBox">
       <property name="standardButtons">
        <set>QDialogButtonBox::Ok
             |QDialogButtonBox::Cancel</set>
       </property>
      </widget>
     </item>
    </layout>
   </widget>
  </ui>
```

Adım 5.3: Ayarlar Penceresini MainWindow'a Bağlama

`main_window.h` dosyasına ekleyin:

```
// private slots bolumune ekleyin:
    void disaAktarJson();
    void ayarlariGoster();
```

`main_window.cpp` dosyasına `#include "ui_ayarlar_dialog.h"` ve `#include "json_exporter.h"` ekledikten sonra:

```
void MainWindow::disaAktarJson() {
    QString yol =
        QFileDialog::getSaveFileName(
            this, "JSON Aktar", "obys.json",
            "JSON (*.json)");
    if (yol.isEmpty()) return;
    if (!yol.endsWith(".json",
            Qt::CaseInsensitive))
        yol += ".json";

    JsonExporter dis;
    if (!dis.disaAktar(yol, m_servis)) {
        QMessageBox::warning(this, "Hata",
            dis.sonHata());
        return;
    }
    statusBar()->showMessage(
        "JSON: " + yol, 3000);
}

void MainWindow::ayarlariGoster() {
    QDialog dlg(this);
    Ui::AyarlarDialog ui;
    ui.setupUi(&dlg);

    // Mevcut ayarlari yukle
    QSettings s;
    const QString tema =
        s.value("gorunum/tema", "light")
         .toString();
    ui.aydinlikRadio->setChecked(
        tema == "light");
    ui.karanlikRadio->setChecked(
        tema == "dark");
    ui.otomatikKaydetSpin->setValue(
        s.value("kayit/otomatikDakika", 5)
         .toInt());
```

```
    ui.sonAcilanSayiSpin->setValue(
        s.value("dosya/maxSonAcilan", 5)
         .toInt());

    connect(ui.buttonBox,
            &QDialogButtonBox::accepted,
            &dlg, &QDialog::accept);
    connect(ui.buttonBox,
            &QDialogButtonBox::rejected,
            &dlg, &QDialog::reject);

    if (dlg.exec() != QDialog::Accepted) return;

    // Ayarlari kaydet
    const QString yeniTema =
        ui.karanlikRadio->isChecked()
            ? "dark" : "light";
    s.setValue("gorunum/tema", yeniTema);
    s.setValue("kayit/otomatikDakika",
               ui.otomatikKaydetSpin->value());
    s.setValue("dosya/maxSonAcilan",
               ui.sonAcilanSayiSpin->value());

    statusBar()->showMessage(
        "Ayarlar kaydedildi", 2000);
}
```

Adım 5.4: Menüleri Güncelleyin

`menuleriKur()` içinde “Dosya” menüsüne JSON aktarımı, ““Veri” menüsüne Ayarlar ekleyin:

```
// Dosya menusune (Kaydet'ten sonra):
dosyaMenu->addSeparator();
auto* disaAktarMenu =
    dosyaMenu->addMenu("Dışa Aktar");
auto* jsonAct =
    disaAktarMenu->addAction("JSON...");
jsonAct->setShortcut(
    QKeySequence(Qt::CTRL | Qt::SHIFT
                 | Qt::Key_E));
connect(jsonAct, &QAction::triggered,
        this, &MainWindow::disaAktarJson);

// Veri menusune (en sona):
veriMenu->addSeparator();
auto* ayarlarAct =
    veriMenu->addAction("Ayarlar...");
connect(ayarlarAct, &QAction::triggered,
        this, &MainWindow::ayarlariGoster);
```

Adım 5.5: CMakeLists.txt Güncelleme

```
# OBYS_SOURCES listesine ekleyin:
    src/json_exporter.cpp    src/json_exporter.h
    ui/ayarlar_dialog.ui
```

Görev 5.1 – Sorular (5 dk)

a) `QSaveFile` ile `QFile` arasındaki fark nedir? Neden JSON yazarken `QSaveFile` tercih edilmiştir?

_____

b) `overloaded` helper struct'ı ne işe yarar? C++17'nin hangi özelliğini kullanır?

_____

c) Ayarlar penceresindeki "Otomatik Kaydetme" değeri `QSettings`'e nasıl kaydediliyor? Bu değer uygulamanın hangi oturumlarında kalıcıdır?

_____

Kontrol Listesi – Aşama 5

- ☐ `JsonExporter` sınıfı tamamlandı
- ☐ "Dışa Aktar → JSON" menüsü çalışıyor
- ☐ Oluşturulan JSON dosyası geçerli ve okunabilir
- ☐ Ayarlar penceresi (.ui) oluşturuldu ve açılıyor
- ☐ Tema, otomatik kaydetme ve son açılanlar ayarları kaydediliyor
- ☐ `CMakeLists.txt` güncellendi
- ☐ Proje derleniyor ve çalışıyor

7 Aşama 6: Grafikler ve Raporlar (60 dk)

Ön Bilgi

QtCharts modülü ile veri görselleştirme yapacağız. Harf notu dağılımını çubuk grafik olarak göstermek için `IstatistikModel` (histogram proxy modeli) ve `GrafikPaneli` (grafik widget) sınıflarını oluşturacağız. Ayrıca `TranskriptUretici` ile HTML tabanlı transkript ve `DersRaporu` ile çok sayfalı rapor üretimi ekleyeceğiz.

Adım 6.1: IstatistikModel (Histogram Proxy)

`src/istatistik_model.h`:

```
#ifndef ISTATISTIK_MODEL_H
#define ISTATISTIK_MODEL_H

#include <QAbstractTableModel>
#include <QPointer>
#include <QStringList>
#include <QVector>

class IstatistikModel
    : public QAbstractTableModel {
    Q_OBJECT

public:
    explicit IstatistikModel(
        int harfSutunu = 3,
        QObject* parent = nullptr);

    void setKaynakModel(
        QAbstractItemModel* kaynak);

    int rowCount(const QModelIndex& parent
        = QModelIndex()) const override;
    int columnCount(const QModelIndex& parent
        = QModelIndex()) const override;
    QVariant data(const QModelIndex& idx,
        int role = Qt::DisplayRole)
        const override;
    QVariant headerData(int section,
        Qt::Orientation orient,
        int role = Qt::DisplayRole)
        const override;

private slots:
    void kaynakDegisti();

private:
    void histogramiHesapla();

    QPointer<QAbstractItemModel> m_kaynak;
    int          m_harfSutunu;
    QStringList  m_harfler;
    QVector<int> m_sayilar;
};

```

```
#endif // ISTATISTIK_MODEL_H
```

src/istatistik_model.cpp:

```
#include "istatistik_model.h"

IstatistikModel::IstatistikModel(
    int harfSutunu, QObject* parent)
    : QAbstractTableModel(parent)
    , m_harfSutunu(harfSutunu)
    , m_harfler{"AA","BA","BB","CB",
                "CC","DC","DD","FF"}
    , m_sayilar(8, 0)
{}

void IstatistikModel::setKaynakModel(
    QAbstractItemModel* kaynak) {
    if (m_kaynak)
        disconnect(m_kaynak, nullptr,
                   this, nullptr);

    m_kaynak = kaynak;

    if (m_kaynak) {
        connect(m_kaynak,
            &QAbstractItemModel::dataChanged,
            this, &IstatistikModel::kaynakDegisti);
        connect(m_kaynak,
            &QAbstractItemModel::modelReset,
            this, &IstatistikModel::kaynakDegisti);
        connect(m_kaynak,
            &QAbstractItemModel::rowsInserted,
            this, &IstatistikModel::kaynakDegisti);
        connect(m_kaynak,
            &QAbstractItemModel::rowsRemoved,
            this, &IstatistikModel::kaynakDegisti);
    }

    beginResetModel();
    histogramiHesapla();
    endResetModel();
}

int IstatistikModel::rowCount(
    const QModelIndex& parent) const {
    return parent.isValid()
        ? 0 : m_harfler.size();
}

int IstatistikModel::columnCount(
    const QModelIndex& parent) const {
    return parent.isValid() ? 0 : 2;
}

QVariant IstatistikModel::data(
    const QModelIndex& idx, int role) const {
    if (!idx.isValid()) return {};
    if (role != Qt::DisplayRole
        && role != Qt::EditRole) return {};
    const int r = idx.row();
```

```
    if (r < 0 || r >= m_harfler.size()) return {};
    return idx.column() == 0
        ? QVariant(m_harfler.at(r))
        : QVariant(m_sayilar.value(r, 0));
}

QVariant IstatistikModel::headerData(
    int section, Qt::Orientation orient,
    int role) const {
    if (role != Qt::DisplayRole) return {};
    if (orient == Qt::Horizontal) {
        return section == 0 ? "Harf" : "Sayi";
    }
    return section + 1;
}

void IstatistikModel::kaynakDegisti() {
    histogramiHesapla();
    const QModelIndex sol = index(0, 1);
    const QModelIndex sag =
        index(rowCount() - 1, 1);
    emit dataChanged(sol, sag,
                     {Qt::DisplayRole});
}

void IstatistikModel::histogramiHesapla() {
    m_sayilar.fill(0, 8);
    if (!m_kaynak) return;
    for (int r = 0;
         r < m_kaynak->rowCount(); ++r) {
        const QString harf =
            m_kaynak->data(
                m_kaynak->index(r,
                    m_harfSutunu))
            .toString().trimmed().toUpper();
        const int idx =
            m_harfler.indexOf(harf);
        if (idx >= 0) ++m_sayilar[idx];
    }
}
```

Adım 6.2: GrafikPaneli

src/grafik_paneli.h:

```
#ifndef GRAFIK_PANELI_H
#define GRAFIK_PANELI_H

#include <QWidget>

class QAbstractItemModel;
class QChart;
class QChartView;
class QBarSeries;
class QBarCategoryAxis;
class QValueAxis;
class QVBarModelMapper;

class GrafikPaneli : public QWidget {
```

```
    Q_OBJECT
public:
    explicit GrafikPaneli(
        QWidget* parent = nullptr);

    void setModel(
        QAbstractItemModel* model);

private:
    QChart*             m_grafik{nullptr};
    QChartView*         m_gorunum{nullptr};
    QBarSeries*         m_seri{nullptr};
    QBarCategoryAxis*   m_xEksen{nullptr};
    QValueAxis*         m_yEksen{nullptr};
    QVBarModelMapper*   m_mapper{nullptr};
};

#endif // GRAFIK_PANELI_H
```

src/grafik_paneli.cpp:

```
#include "grafik_paneli.h"

#include <QBarCategoryAxis>
#include <QBarSeries>
#include <QBarSet>
#include <QChart>
#include <QChartView>
#include <QVBarModelMapper>
#include <QValueAxis>
#include <QVBoxLayout>
#include <QPainter>

GrafikPaneli::GrafikPaneli(QWidget* parent)
    : QWidget(parent) {
    m_seri = new QBarSeries(this);
    m_seri->setLabelsVisible(true);

    m_grafik = new QChart();
    m_grafik->addSeries(m_seri);
    m_grafik->setTitle("Harf Notu Dagilimi");
    m_grafik->setAnimationOptions(
        QChart::SeriesAnimations);
    m_grafik->legend()->setVisible(true);
    m_grafik->legend()->setAlignment(
        Qt::AlignBottom);

    QStringList harfler{
        "AA","BA","BB","CB",
        "CC","DC","DD","FF"};
    m_xEksen = new QBarCategoryAxis();
    m_xEksen->append(harfler);
    m_grafik->addAxis(m_xEksen,
                      Qt::AlignBottom);
    m_seri->attachAxis(m_xEksen);

    m_yEksen = new QValueAxis();
    m_yEksen->setRange(0, 10);
    m_yEksen->setTickCount(11);
    m_yEksen->setLabelFormat("%d");
```

```
    m_yEksen->setTitleText("Ogrenci Sayisi");
    m_grafik->addAxis(m_yEksen,
                      Qt::AlignLeft);
    m_seri->attachAxis(m_yEksen);

    m_gorunum = new QChartView(
        m_grafik, this);
    m_gorunum->setRenderHint(
        QPainter::Antialiasing);

    auto* lay = new QVBoxLayout(this);
    lay->setContentsMargins(0, 0, 0, 0);
    lay->addWidget(m_gorunum);
}

void GrafikPaneli::setModel(
    QAbstractItemModel* model) {
    delete m_mapper;
    m_mapper = nullptr;
    m_seri->clear();
    if (!model) return;

    m_mapper = new QVBarModelMapper(this);
    m_mapper->setFirstBarSetColumn(1);
    m_mapper->setLastBarSetColumn(1);
    m_mapper->setFirstRow(0);
    m_mapper->setRowCount(8);
    m_mapper->setSeries(m_seri);
    m_mapper->setModel(model);

    if (!m_seri->barSets().isEmpty())
        m_seri->barSets().first()->setLabel(
            "Ogrenci Sayisi");
}
```

Adım 6.3: TranskriptUretici (HTML Rapor)

`src/transkript_uretici.h`:

```
#ifndef TRANSKRIPT_URETICI_H
#define TRANSKRIPT_URETICI_H

#include "obys_domain.h"
#include <QList>
#include <QString>

class TranskriptUretici {
public:
    QString html(
        const Ogrenci& ogrenci,
        const QList<Ders>& dersler,
        const QList<DersKaydi>& kayitlar) const;

private:
    static double gpaKatsayisi(
        const NotTuru& n);
    static QString notMetni(
        const NotTuru& n);
    static QString sablonSar(
```

```
        const QString& govde);
};

#endif // TRANSKRIPT_URETICI_H
```

src/transkript_uretici.cpp:

```
#include "transkript_uretici.h"

double TranskriptUretici::gpaKatsayisi(
    const NotTuru& n) {
    double k = 0.0;
    std::visit([&k](const auto& d) {
        using T = std::decay_t<decltype(d)>;
        if constexpr (
            std::is_same_v<T, SayisalNot>) {
            double s = d.deger;
            if      (s >= 90) k = 4.0;
            else if (s >= 85) k = 3.5;
            else if (s >= 80) k = 3.0;
            else if (s >= 75) k = 2.5;
            else if (s >= 70) k = 2.0;
            else if (s >= 60) k = 1.5;
            else if (s >= 50) k = 1.0;
        } else if constexpr (
            std::is_same_v<T, HarfNotu>) {
            QString h = d.harf.toUpper();
            if      (h == "AA") k = 4.0;
            else if (h == "BA") k = 3.5;
            else if (h == "BB") k = 3.0;
            else if (h == "CB") k = 2.5;
            else if (h == "CC") k = 2.0;
            else if (h == "DC") k = 1.5;
            else if (h == "DD") k = 1.0;
        }
    }, n);
    return k;
}

QString TranskriptUretici::notMetni(
    const NotTuru& n) {
    QString s;
    std::visit([&s](const auto& d) {
        using T = std::decay_t<decltype(d)>;
        if constexpr (
            std::is_same_v<T, SayisalNot>)
            s = QString::number(d.deger, 'f', 2);
        else if constexpr (
            std::is_same_v<T, HarfNotu>)
            s = d.harf;
        else if constexpr (
            std::is_same_v<T, MuafNot>)
            s = "MUAF";
    }, n);
    return s;
}

QString TranskriptUretici::sablonSar(
    const QString& govde) {
    return QString(
```

```
        "<html><head>"
        "<meta charset=\"utf-8\"/>"
        "<style>"
        "body{font-family:sans-serif;}"
        "h1{color:#1b4f72;}"
        "table{border-collapse:collapse;"
        "width:100%;}"
        "th,td{border:1px solid #888;"
        "padding:4px 8px;}"
        "th{background:#d6eaf8;}"
        ".ozet{margin-top:12px;"
        "font-weight:bold;}"
        "</style></head><body>")
        + govde + "</body></html>";
}

QString TranskriptUretici::html(
    const Ogrenci& ogr,
    const QList<Ders>& dersler,
    const QList<DersKaydi>& kayitlar) const {
    QString g;
    g += "<h1>Ogrenci Transkripti</h1>";
    g += QString(
        "<p><b>No:</b> %1<br/>"
        "<b>Ad Soyad:</b> %2</p>")
        .arg(ogr.numara.toHtmlEscaped(),
             ogr.adSoyad.toHtmlEscaped());

    g += "<table><tr>"
         "<th>Kod</th><th>Ders</th>"
         "<th>Kredi</th><th>Not</th></tr>";

    double topAgirlik = 0;
    int    topKredi   = 0;

    for (const auto& k : kayitlar) {
        if (k.ogrenciId != ogr.id) continue;
        const Ders* dp = nullptr;
        for (const auto& d : dersler)
            if (d.id == k.dersId)
                { dp = &d; break; }
        if (!dp) continue;

        g += QString(
            "<tr><td>%1</td><td>%2</td>"
            "<td>%3</td><td>%4</td></tr>")
            .arg(dp->kod.toHtmlEscaped(),
                 dp->ad.toHtmlEscaped(),
                 QString::number(dp->kredi),
                 notMetni(k.notu)
                     .toHtmlEscaped());

        bool muaf =
            std::holds_alternative<MuafNot>(
                k.notu);
        if (!muaf && dp->kredi > 0) {
            topAgirlik +=
                gpaKatsayisi(k.notu)
                * dp->kredi;
```

```
            topKredi += dp->kredi;
        }
    }

    g += "</table>";
    double gpa = topKredi > 0
        ? topAgirlik / topKredi : 0.0;
    g += QString(
        "<p class=\"ozet\">"
        "Toplam Kredi: %1   GPA: %2</p>")
        .arg(QString::number(topKredi),
             QString::number(gpa, 'f', 2));

    return sablonSar(g);
}
```

Adım 6.4: Grafik ve Raporları Ana Pencereye Entegre Etme

`main_window.h` dosyasına eklemeler:

```
// Include'lar:
class IstatistikModel;
class GrafikPaneli;

// Private uye degiskenler:
    IstatistikModel* m_istatistikModel{nullptr};
    GrafikPaneli*    m_grafikPanel{nullptr};

// Private slots:
    void transkriptYazdir();
```

`main_window.cpp` dosyasında `merkeziWidgetKur()` fonksiyonunu güncelleyin:

```
// Include'lara ekleyin:
#include "grafik_paneli.h"
#include "istatistik_model.h"
#include "transkript_uretici.h"
#include <QInputDialog>
#include <QTextDocument>
#include <QtPrintSupport/QPrintPreviewDialog>
#include <QtPrintSupport/QPrinter>

// merkeziWidgetKur() guncelle:
void MainWindow::merkeziWidgetKur() {
    m_sekmeler = new QTabWidget(this);

    m_sekmeler->addTab(
        notlarSekmesiKur(), "Notlar");

    m_grafikPanel = new GrafikPaneli(this);
    m_istatistikModel =
        new IstatistikModel(3, this);
    m_istatistikModel->setKaynakModel(
        m_notlarModel);
    m_grafikPanel->setModel(
        m_istatistikModel);
    m_sekmeler->addTab(
        m_grafikPanel, "Grafikler");

    setCentralWidget(m_sekmeler);
```

```
}

// Transkript yazdir:
void MainWindow::transkriptYazdir() {
    const auto ogrenciler =
        m_servis.tumOgrenciler();
    if (ogrenciler.isEmpty()) {
        QMessageBox::information(this,
            "Transkript", "Liste boş.");
        return;
    }

    QStringList isimler;
    for (const auto& o : ogrenciler)
        isimler << QString("%1 - %2")
                        .arg(o.numara, o.adSoyad);

    bool ok = false;
    const QString secim =
        QInputDialog::getItem(
            this, "Transkript",
            "Öğrenci seçin:", isimler,
            0, false, &ok);
    if (!ok) return;

    const int idx = isimler.indexOf(secim);
    if (idx < 0) return;
    const Ogrenci& ogr = ogrenciler.at(idx);

    TranskriptUretici uretici;
    auto* belge = new QTextDocument();
    belge->setHtml(uretici.html(
        ogr, m_servis.tumDersler(),
        m_servis.tumNotlar()));

    QPrinter yazici(QPrinter::HighResolution);
    QPrintPreviewDialog onizleme(
        &yazici, this);
    connect(&onizleme,
        &QPrintPreviewDialog::paintRequested,
        this, [belge](QPrinter* p) {
            belge->print(p);
        });
    onizleme.exec();
    belge->deleteLater();
}
```

Menüye transkript aksiyonunu ekleyin (Dosya menüsünde Çıkış'tan önce):

```
dosyaMenu->addSeparator();
auto* transkriptAct =
    dosyaMenu->addAction("Transkript...");
transkriptAct->setShortcut(QKeySequence::Print);
connect(transkriptAct, &QAction::triggered,
        this, &MainWindow::transkriptYazdir);
```

Adım 6.5: CMakeLists.txt Güncelleme

```
# OBYS_SOURCES listesine ekleyin:
```

```
src/istatistik_model.cpp src/istatistik_model.h
src/grafik_paneli.cpp    src/grafik_paneli.h
src/transkript_uretici.cpp
src/transkript_uretici.h
```

Görev 6.1 – Sorular (10 dk)

a) `QVBarModelMapper` ne işe yarar? Model verisi değiştiğinde grafik nasıl güncellenir?

_____

b) `IstatistikModel`, `NotlarModel`'in hangi sinyallerini dinler? Bu sinyallerden hangisi `yenile()` çağrıldığında tetiklenir?

_____

c) GPA hesabında "muaf" notlar neden hariç tutuluyor? `std::holds_alternative<MuafNot>()` ne döndürür?

_____

d) Transkript HTML olarak üretilip `QTextDocument` ile basılıyor. Bu yaklaşımın doğrudan `QPainter` kullanmaya göre avantajı nedir?

_____

Kontrol Listesi – Aşama 6

- ☐ "Grafikler" sekmesinde çubuk grafik görünüyor
- ☐ Not eklendikçe histogram otomatik güncelleniyor
- ☐ "Dosya → Transkript" ile önizleme açılıyor
- ☐ Transkript'te öğrenci bilgileri, dersler ve GPA doğru
- ☐ `IstatistikModel` doğru saydığını doğruladınız
- ☐ Proje derleniyor

8 Aşama 7: Tema Desteği, QSettings ve Son Dokunuşlar (45 dk)

Ön Bilgi

Son aşamada uygulamaya aydınlık/karanlık tema desteği, QSS stil dosyaları ve Görünüm / Yardım menüleri ekleyeceğiz. Tüm kullanıcı tercihleri `QSettings` ile kalıcı hale getirilecektir.

Adım 7.1: Tema QSS Dosyaları

`resources/themes/light.qss` dosyasını oluşturun:

```
QMainWindow {
    background-color: #f7f7f7;
}
QMenuBar {
    background-color: #ffffff;
    color: #222222;
    border-bottom: 1px solid #e0e0e0;
}
QMenuBar::item:selected {
    background-color: #e6f0fa;
}
QMenu {
    background-color: #ffffff;
    border: 1px solid #cccccc;
}
QMenu::item:selected {
    background-color: #e6f0fa;
}
QPushButton {
    background-color: #ffffff;
    border: 1px solid #bbbbbb;
    border-radius: 4px;
    padding: 6px 14px;
}
QPushButton:hover {
    background-color: #e6f0fa;
    border-color: #4a90e2;
}
QTableView {
    background-color: #ffffff;
    alternate-background-color: #f2f6fb;
    selection-background-color: #4a90e2;
    selection-color: #ffffff;
    gridline-color: #dddddd;
}
QHeaderView::section {
    background-color: #eeeeee;
    padding: 4px;
    border: 1px solid #cccccc;
}
QTabWidget::pane {
    border: 1px solid #cccccc;
    background: #ffffff;
}
QTabBar::tab {
    background: #eeeeee;
    padding: 6px 14px;
```

```
    border: 1px solid #cccccc;
}
QTabBar::tab:selected {
    background: #ffffff;
    border-bottom-color: #ffffff;
}
QStatusBar {
    background-color: #eeeeee;
    color: #555555;
}
```

`resources/themes/dark.qss` dosyasını oluşturun:

```
QMainWindow, QWidget {
    background-color: #2b2b2b;
    color: #e0e0e0;
}
QMenuBar {
    background-color: #3c3c3c;
    color: #e0e0e0;
    border-bottom: 1px solid #1e1e1e;
}
QMenuBar::item:selected {
    background-color: #4a90e2;
    color: #ffffff;
}
QMenu {
    background-color: #3c3c3c;
    color: #e0e0e0;
    border: 1px solid #1e1e1e;
}
QMenu::item:selected {
    background-color: #4a90e2;
    color: #ffffff;
}
QPushButton {
    background-color: #3c3c3c;
    border: 1px solid #555555;
    border-radius: 4px;
    padding: 6px 14px;
    color: #e0e0e0;
}
QPushButton:hover {
    background-color: #4a90e2;
    color: #ffffff;
}
QTableView {
    background-color: #1e1e1e;
    alternate-background-color: #262626;
    selection-background-color: #4a90e2;
    selection-color: #ffffff;
    gridline-color: #444444;
    color: #e0e0e0;
}
QHeaderView::section {
    background-color: #3c3c3c;
    color: #e0e0e0;
    padding: 4px;
    border: 1px solid #1e1e1e;
}
```

```
QTabWidget::pane {
    border: 1px solid #1e1e1e;
    background: #2b2b2b;
}
QTabBar::tab {
    background: #3c3c3c;
    color: #e0e0e0;
    padding: 6px 14px;
    border: 1px solid #1e1e1e;
}
QTabBar::tab:selected {
    background: #4a90e2;
    color: #ffffff;
}
QStatusBar {
    background-color: #3c3c3c;
    color: #cccccc;
}
QLineEdit, QComboBox, QSpinBox,
QDoubleSpinBox {
    background-color: #1e1e1e;
    color: #e0e0e0;
    border: 1px solid #555555;
    border-radius: 3px;
    padding: 3px;
}
```

Adım 7.2: QRC Kaynak Dosyası

`CMakeLists.txt` dosyasına tema kaynaklarını ekleyin:

```
qt_add_resources(obys_tutorial "obys_res"
    PREFIX "/"
    FILES
        resources/themes/light.qss
        resources/themes/dark.qss
)
```

Adım 7.3: Tema Uygulama ve Görünüm Menüsü

`main_window.h` dosyasına ekleyin:

```
// private slots:
    void temayiUygula(const QString& temaAd);
    void hakkindaGoster();
```

`main_window.cpp`:

```
#include <QActionGroup>
#include <QFile>
#include <QTextStream>

void MainWindow::temayiUygula(
    const QString& temaAd) {
    const QString yol =
        (temaAd == "dark")
            ? ":/themes/dark.qss"
            : ":/themes/light.qss";

```

```
    QFile dosya(yol);
    QString qss;
    if (dosya.open(QIODevice::ReadOnly
                   | QIODevice::Text)) {
        QTextStream akis(&dosya);
        qss = akis.readAll();
    }
    qApp->setStyleSheet(qss);

    QSettings s;
    s.setValue("gorunum/tema", temaAd);
    statusBar()->showMessage(
        "Tema: " + temaAd, 1500);
}

void MainWindow::hakkindaGoster() {
    QMessageBox::about(this, "Hakkinda",
        "<h3>OBYS Tutorial v1.0</h3>"
        "<p>Ogrenci Bilgi Yonetim Sistemi</p>"
        "<p>KTU Bilgisayar Bilimleri - "
        "Yazilim Gelistirme II</p>");
}
```

`menuleriKur()` fonksiyonuna Görünüm ve Yardım menülerini ekleyin:

```
// menuleriKur() icerisine:

// Gorunum menusu
auto* gorunumMenu =
    menuBar()->addMenu("Görünüm");

auto* temaGrup = new QActionGroup(this);
temaGrup->setExclusive(true);

auto* aydinlikAct =
    gorunumMenu->addAction("Aydinlik Tema");
aydinlikAct->setCheckable(true);
aydinlikAct->setShortcut(
    QKeySequence(Qt::CTRL | Qt::Key_1));
temaGrup->addAction(aydinlikAct);
connect(aydinlikAct, &QAction::triggered,
    this, [this]{ temayiUygula("light"); });

auto* karanlikAct =
    gorunumMenu->addAction("Karanlik Tema");
karanlikAct->setCheckable(true);
karanlikAct->setShortcut(
    QKeySequence(Qt::CTRL | Qt::Key_2));
temaGrup->addAction(karanlikAct);
connect(karanlikAct, &QAction::triggered,
    this, [this]{ temayiUygula("dark"); });

// Mevcut tema durumunu yansit
QSettings s;
const QString tema =
    s.value("gorunum/tema", "light").toString();
aydinlikAct->setChecked(tema == "light");
karanlikAct->setChecked(tema == "dark");

// Yardim menusu
```

```
auto* yardimMenu =
    menuBar()->addMenu("Yardim");
auto* hakkindaAct =
    yardimMenu->addAction("Hakkinda...");
connect(hakkindaAct, &QAction::triggered,
        this, &MainWindow::hakkindaGoster);
```

Adım 7.4: main.cpp – Başlangıçta Tema Yükleme

```
#include "main_window.h"
#include <QApplication>
#include <QFile>
#include <QSettings>
#include <QTextStream>

int main(int argc, char* argv[]) {
    QApplication uygulama(argc, argv);

    QCoreApplication::setOrganizationName("KTU");
    QCoreApplication::setApplicationName(
        "OBYS Tutorial");
    QCoreApplication::setApplicationVersion(
        "1.0.0");

    // Kayitli temayi yukle
    QSettings ayarlar;
    const QString temaAd =
        ayarlar.value("gorunum/tema", "light")
               .toString();
    const QString qssYol =
        (temaAd == "dark")
            ? ":/themes/dark.qss"
            : ":/themes/light.qss";

    QFile dosya(qssYol);
    if (dosya.open(QIODevice::ReadOnly
                   | QIODevice::Text)) {
        QTextStream akis(&dosya);
        uygulama.setStyleSheet(akis.readAll());
    }

    MainWindow pencere;
    pencere.show();
    return uygulama.exec();
}
```

Görev 7.1 – Sorular (5 dk)

a) QSS (Qt Style Sheet) ile CSS arasındaki benzerlik ve farklılıklar nelerdir?

_____

b) `QActionGroup` ile `setExclusive(true)` ne sağlar? Bu olmadan tema menüsü nasıl davranırdı?

_____

c) Tema dosyaları neden `QRC` içine gömülüyor? Dosya sisteminden okunması yerine QRC tercih edilmesinin avantajı nedir?

_____

Kontrol Listesi – Aşama 7

- ☐ `resources/themes/light.qss` ve `dark.qss` oluşturuldu
- ☐ QRC kaynakları `CMakeLists.txt`'ye eklendi
- ☐ "Görünüm" menüsünde Aydınlık/Karanlık Tema seçilebiliyor
- ☐ Tema değişikliği anında uygulanıyor
- ☐ Tema tercihi `QSettings` ile kalıcı hale geliyor
- ☐ Uygulama yeniden başlatıldığında son tema yükleniyor
- ☐ "Yardım → Hakkında" diyalogu açılıyor
- ☐ Tüm menüler ve kısayollar çalışıyor

Final CMakeLists.txt (Referans)

Tüm aşamalar tamamlandığında `CMakeLists.txt` dosyanız şöyle olmalıdır:

```
cmake_minimum_required(VERSION 3.21)

project(obys_tutorial
    VERSION 1.0.0
    DESCRIPTION "OBYS Tutorial"
    LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)
set(CMAKE_AUTOUIC ON)

find_package(Qt6 6.5 REQUIRED COMPONENTS
    Widgets PrintSupport Charts)

qt_standard_project_setup()

set(OBYS_SOURCES
    src/main.cpp
    src/main_window.cpp
    src/main_window.h
    src/persistence_manager.cpp
    src/persistence_manager.h
    src/not_giris_dialog.cpp
    src/not_giris_dialog.h
    src/notlar_model.cpp
    src/notlar_model.h
    src/istatistik_model.cpp
    src/istatistik_model.h
    src/grafik_paneli.cpp
    src/grafik_paneli.h
    src/transkript_uretici.cpp
    src/transkript_uretici.h
    src/json_exporter.cpp
    src/json_exporter.h
    ui/ogrenci_ekle_dialog.ui
    ui/ders_ekle_dialog.ui
    ui/ayarlar_dialog.ui
)

qt_add_executable(obys_tutorial ${OBYS_SOURCES})

qt_add_resources(obys_tutorial "obys_res"
    PREFIX "/"
    FILES
        resources/themes/light.qss
        resources/themes/dark.qss
)

target_include_directories(obys_tutorial
    PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}/include
```

```
        ${CMAKE_CURRENT_SOURCE_DIR}/src
        ${CMAKE_CURRENT_SOURCE_DIR}/ui
)

target_link_libraries(obys_tutorial
    PRIVATE
        Qt6::Widgets
        Qt6::PrintSupport
        Qt6::Charts
)

if(MSVC)
    target_compile_options(obys_tutorial
        PRIVATE /W4)
else()
    target_compile_options(obys_tutorial
        PRIVATE -Wall -Wextra -Wpedantic)
endif()
```

Genel Değerlendirme Soruları

Tüm aşamaları tamamladıktan sonra aşağıdaki soruları yanıtlayın:

Değerlendirme (15 dk)

1) Projenin katmanlı mimarisini (domain → servis → UI) çizin. Her katmanın sorumluluğunu bir cümleyle açıklayın.

2) `std::variant` + `std::visit` deseni yerine kalıtım (inheritance) kullansaydınız hangi sınıfları oluşturmanız gerekirdi? Her iki yaklaşımın artı ve eksilerini karşılaştırın.

3) Model/View mimarisi (QAbstractTableModel + QTableView) doğrudan widget'a veri yazmaya göre ne avantaj sağlar? En az 3 avantaj yazın.

4) `QSettings` verilerini hangi platformda nerede saklar? (Windows, macOS, Linux için ayrı ayrı belirtin.)

5) Uygulamaya "Öğrenci Silme" özelliği eklemek isteseydiniz hangi dosyaları değiştirmeniz gerekirdi? Adımları listeleyiniz.

6) Bu projeye veritabanı (SQLite) desteği eklemek isteseydiniz mimaride hangi değişiklikleri yapardınız? `OBYSServis` sınıfının rolü nasıl değişirdi?

Final Proje Dosya Yapısı

```
obys_tutorial/
  CMakeLists.txt
  include/
    obys_domain.h
    obys_servis.h
  src/
    main.cpp
    main_window.h           main_window.cpp
    persistence_manager.h   persistence_manager.cpp
    not_giris_dialog.h      not_giris_dialog.cpp
    notlar_model.h          notlar_model.cpp
    istatistik_model.h      istatistik_model.cpp
    grafik_paneli.h         grafik_paneli.cpp
    transkript_uretici.h    transkript_uretici.cpp
    json_exporter.h         json_exporter.cpp
  ui/
    ogrenci_ekle_dialog.ui
    ders_ekle_dialog.ui
    ayarlar_dialog.ui
  resources/
    themes/
      light.qss
      dark.qss
```

Teslim Bilgileri

- Projenin tamamını ZIP olarak sıkıştırıp teslim edin
- Tüm kontrol listeleri ve soruların yanıtlanmış olması gerekir
- Proje `cmake -B build && cmake -build build` ile hatasız derlenmeli ve çalışmalıdır

Puanlama:

Aşama	Puan
Aşama 0: Proje İskeleti	5
Aşama 1: Veri Sınıfları	15
Aşama 2: Yardımcı Sınıflar	10
Aşama 3: Veri Giriş Formları	15
Aşama 4: Ana Pencere	20
Aşama 5: DTO, JSON ve Ayarlar	10
Aşama 6: Grafikler ve Raporlar	15
Aşama 7: Tema ve Son Dokunuşlar	5
Sorular (tüm aşamalar)	5
Toplam	**100**